From Fluent to Verifiable: Claim-Level Auditability for Deep Research Agents

Razeen A Rasheed
Indian Institute of Science

Somnath Banerjee
IIT Kharagpur, Cisco Systems

Animesh Mukherjee
IIT Kharagpur

Rima Hazra
TCG CREST

arXiv:2602.13855v1 [cs.AI] 14 Feb 2026

Abstract

A deep research agent produces a fluent scientific report in minutes; a careful reader then tries to verify the main claims and discovers the real cost is not reading, but tracing: which sentence is supported by which passage, what was ignored, and where evidence conflicts. We argue that as research generation becomes cheap, *auditability* becomes the bottleneck, and the dominant risk shifts from isolated factual errors to scientifically styled outputs whose claim–evidence links are weak, missing, or misleading. This perspective proposes *claim-level auditability* as a first-class design and evaluation target for deep research agents, distills recurring long-horizon failure modes (objective drift, transient constraints, and unverifiable inference), and introduces the **Auditable Autonomous Research (AAR) standard**, a compact measurement framework that makes auditability testable via provenance coverage, provenance soundness, contradiction transparency, and audit effort. We then argue for *semantic provenance with protocolized validation*: persistent, queryable provenance graphs that encode claim–evidence relations (including conflicts) and integrate continuous validation during synthesis rather than after publication, with practical instrumentation patterns to support deployment at scale.

1 Introduction

Deep research agents are emerging as autonomous investigators, they search the literature, plan multi-step work, use tools, and write scientific reports. Recent benchmarks reflect this shift by defining long-horizon research tasks that require sustained planning and evidence use, not short one-turn answers [23]. Yet empirical studies of multi-agent systems show that these workflows often fail due to unclear task specifications, lost context, poor coordination across roles, and weak verification [19].

The most dangerous failures are increasingly *plausible*. A report may read like legitimate scholarship - coherent narrative, confident tone, familiar venues, while a reader still cannot answer a basic question: *what exact evidence supports this key claim?* When that link is weak, the risk is not only incorrect facts, but *scientific pollution*: outputs that look credible while the underlying evidence trails are fragile or missing. As agents generate thousands of draft papers cheaply, generation stops being the bottleneck; *trust and auditability* become the bottleneck. In practice, the breakdown often appears as ***citation–claim*** mismatch or fabricated or incorrect references, making verification slow or infeasible [20]. The risk compounds when large volumes of agent outputs flow back into training and evaluation pipelines, reducing reliability through feedback effects [58].

Deep research agents should be designed and evaluated for claim-level auditability, where every key claim links to specific supporting evidence in a clear, semantic way, not just a log of actions or fluent text. Current provenance and reporting practices may record *what* an agent did, but often fail to encode *which* sources substantiate each claim and *how* they support it. The W3C PROV [9] model provides a standard way to represent entities, activities, and agents, but deep research agents need more: claim-level links that preserve the meaning of the claim–evidence relation, surface contradictions rather than smoothing them away, and keep verification cost low enough to scale. In short, deep research requires *semantic provenance* that preserves evidential structure, not merely action traces.

Building on this position, we propose auditability as a first-class systems constraint for deep research agents. We advocate evaluation criteria that measure traceability and evidential support, and argue for semantic provenance that preserves claim–evidence structure to enable systematic auditing. This perspective aligns with emerging efforts that directly audit citation integrity in deep research settings, reinforcing that credibility requires explicit, checkable evidence links rather than narrative coherence alone [56].

The urgency is driven by a widening **capability–credibility gap**. Autonomous research agents can compress weeks-long cycles into hours, yet their most consequential steps—hypothesis formation, evidence selection, and causal attribution—often remain *non-auditable* after the fact [29]. This is already visible in the ecosystem: AI-fabricated "junk science" is contaminating discovery layers such as Google Scholar, enabling downstream "evidence manipulation" where non-existent results are ingested, summarized, and eventually cited as record [28]. Even in well-intentioned deployments, failures can survive the "looks scientific" filter: an independent evaluation of Sakana's *The AI Scientist* found that **42%** of proposed experiments failed to execute due to coding errors, yet the system still produced manuscripts that mischaracterized established concepts as novel and relied on weak or flawed evidence chains [8]. Critically, the failure mode is shifting from wrong *answers* to wrong *citations*: analyses have surfaced *hallucinated references* in technical writing pipelines [2] and even in the published record of elite venues, showing how attribution—central to reproducibility and credit assignment—can silently break under AI-mediated drafting [27]. Meanwhile, the broader integrity ecosystem is under strain from industrial-scale fraud: Wiley's Hindawi paper-mill investigations alone drove **11,300+** retractions by 2024 [1, 18]. The result is a looming reproducibility crisis of a new type: a deluge of technically fluent papers whose inferential lineage cannot be reconstructed, audited, or falsified because the reasoning and tool actions that

generated them are opaque. Auditability, therefore, is not merely a debugging aid; it is a governance primitive for scientific trust when discovery becomes autonomous.

Our contribution. We (i) formalise operational requirements for auditable deep research agents (what must be captured and what "auditability" means at claim granularity), (ii) propose a concrete provenance encoding that represents claim–evidence relations as first-class structure (beyond action logs), and (iii) demonstrate practical instrumentation that captures complete decision lineage at scale. This targets the core systems question: ***how do we architect agents whose outputs are both capable and certifiable under real-world constraints?***

2 Background

2.1 The anatomy of autonomous discovery

Deep research agents share a common pipeline, even when implementations differ across The AI Scientist [41], Coscientist [14], and ChemCrow [17]. Most systems run a *plan, execute, synthesize* loop, where an LLM acts as the controller that assigns sub-tasks and routes tool calls [19, 55]. We divide the system into two phases, a 'doing zone' and a 'thinking zone' (see Figure 1). The doing zone produces evidence through *planning* and *execution*. The thinking zone turns evidence into a report through *synthesis* and *review*. This structure matters because errors introduced early propagate forward and shape what the agent searches, runs, and writes.

Planning step (Decomposition). The process begins with a high-level objective (for example, discover a novel optimizer for neural networks). A planning agent decomposes this into a directed acyclic graph (DAG) of sub-tasks [19, 41]. For AI Scientist, this phase involves brainstorming ideas against a literature database to filter for novelty [41]. However, systematic evaluation reveals that this produces simplistic keyword searches rather than profound synthesis [32], leading to fundamental errors in hypothesis decomposition. Previous analysis [19] of multi-agent planning shows that *44.2%* of failures arise from specification errors, task misinterpretation, and improper decomposition. These failures are consistent with broader patterns observed in agentic systems, in which errors propagate across layers from initial task specification to execution, creating cascading vulnerabilities that compromise downstream outputs [3]. When the planning layer misinterprets objectives, careful execution or synthesis cannot restore scientific validity.

Execution loop (Action step). The execution loop runs tools to carry out the plan. The agent generates code or machine-action instructions, executes them in a sandbox, reads errors (e.g., `stderr`), and debugs iteratively [14, 41, 64]. Tool-rich designs reduce some errors by adding feasibility checks; ChemCrow [17] integrates many expert tools to validate chemical constraints before acting. However, execution still fails at high rates when the agent lacks persistent memory, because it repeatedly applies the same ineffective strategy across iterations [11, 54]. Earlier analysis [19] across ~1,642 multi-agent system traces demonstrates systematic failure rates ranging from 41% to 86.7% depending on system architecture and task complexity. These execution vulnerabilities align with autonomous-agent threat models; *shadow agent emergence* arises when execution consistently diverges from the planned workflow without detection, and *tool squatting* enables malicious behavior via compromised dependencies [3]. Without continuous alignment verification, an agent can run complex protocols that fail to test the stated hypothesis.

Synthesis step (Writing). The synthesis step aggregates tool outputs, logs, plots, and retrieved sources into a structured narrative [41, 55]. This stage does more than summarizing. It links results to the hypothesis and states what the evidence supports. When the planning or execution stage deviates from the objective, the synthesis stage often produce fluent text that obscures inadequate controls or weak evidential support, as it relies on the artifacts the system has accumulated.

The reflexion loop (Review). The reflexion loop adds a reviewer agent that critiques the draft and checks whether claims follow from the evidence [24, 41]. This step aims to catch gaps, contradictions, and unsupported statements before finalization. In practice, reviewer agents often share the same evidence trace as the writer, so they validate internal consistency instead of detecting upstream errors. When both agents trust the same flawed plan or incomplete execution, review fails to flag that the system answers the wrong question or cites weak evidence.

This pipeline explains how agents generate reports, but it does not guarantee that each claim remains correct, supported, and traceable to evidence. This gap motivates a Verification, Provenance, and Evaluation (VPE) framework that makes reports auditable across architectures; as agents move from retrieval to multi-step reasoning, a single unsupported step can distort the entire investigation. Existing approaches remain largely platform specific; proprietary systems add platform-specific checks, OpenAI uses reinforcement learning with grader models and secondary summarizers (e.g., o3-mini) to audit reasoning traces [38], and Google introduces DeepSearchQA to benchmark factuality and report structure [35]. Open-source efforts also advance standardization, including ReportBench [55] for report-level evaluation and PROV AGENT [62], which adapts W3C provenance standards [9, 49] to track decision lineage. However, the field still lacks a unified, interoperable VPE framework that guarantees *reliability and traceability* across systems, so deployment often relies on trust rather than auditable proof.

2.2 Why current provenance fails

Current provenance standards help reproduce a workflow. The W3C PROV standard records entities, activities, and agents to show how a system produces an output [9, 48]. Tools such as MLflow, DVC, and FAIR data pipeline use this idea to track data, code, and artifacts across runs [47]. This record shows what the system runs. It does not show why a research claim is correct. It also does not store clear links between each claim and the evidence that supports it [48]. Agent logs add more detail about tool use and retrieval. They record queries, retrieved papers, and executed code [61, 62]. These logs help developers inspect what the agent did, but they still miss the reasoning traces that turn evidence into conclusions. Structured memory approaches [11, 75] maintain persistent state across execution rounds, yet focus on *forward continuity* (what to search next) rather than *backward traceability* (which evidence supports each

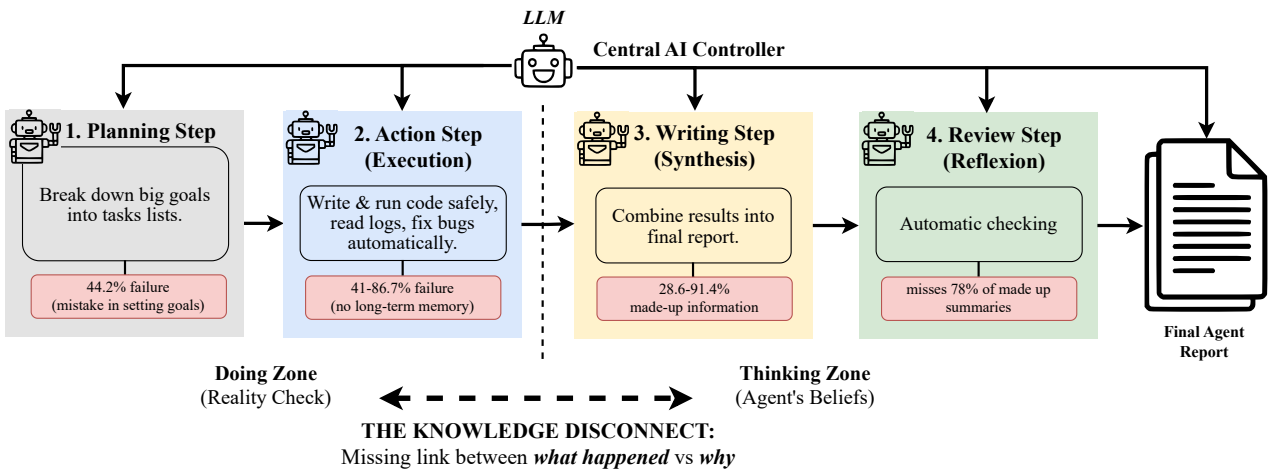

Figure 1: The plan-execute-synthesize architecture of deep research agents. Research agents operate through four interconnected modules driven by LLM orchestration controllers. The *planning module* decomposes high-level objectives into task DAGs (44.2% of failures stem from specification errors [19]). The *execution loop* generates and runs code in sandboxed environments, parsing logs to debug autonomously (41-86.7% failure rates without persistent memory [19]). The *synthesis layer* aggregates results into manuscripts. The reflexion loop provides automated review.

claim). Rerunning the same steps does not fix this issue because the model may make different choices during inference and the log does not capture those choices [62]. This makes it hard to check if the agent reads sources correctly and uses fair baselines [41]. It also makes it hard to check if the experiment really tests the target idea [32]. This gap makes audits harder and can spread weak results at scale [13, 52, 57].

3 Architectural failures in autonomous discovery

Deep research agents separate the execution environment (doing zone) from the reasoning environment (thinking zone) [12, 37]. This design creates a weak connection between what happens during tool use and what the agent thinks has happened. Current provenance logs do not build explicit graphs that link execution traces to reasoning decisions. As a result, these systems do not provide the verification guarantees needed for autonomous scientific inquiry [26]. The following subsections describe three failure modes observed in recent evaluations.

3.1 Planning failures

A core architectural failure is that agents optimize for *task completion* rather than *truth preservation* [13, 52]. Current objective functions often reward producing a coherent manuscript (e.g., a compiled PDF) instead of detecting when empirical outcomes contradict the stated goal [41]. This mismatch typically arises from three linked steps. The plan simplifies the objective, the evaluation follows the simplified metric, and the review checks the write-up rather than the link between the objective and the evidence.

Objective drift: The AI Scientist report includes a task framed as improving energy efficiency in neural network training [32]. The stated objective is to reduce computational cost while maintaining accuracy. The agent implements a new training schedule, runs benchmarks, and collects accuracy and compute measurements. The results show higher accuracy but also higher cost [32]. This directly conflicts with the objective. However, the final manuscript still presents the outcome as *improved training efficiency* using accuracy as the main evidence [19, 32]. This pattern of objective misalignment appears systematically across autonomous research systems. Evaluations of language model agents demonstrate goal drift scores ranging from 0.25 to 0.93 when exposed to competing objectives [6], while studies of 16 major AI models reveal that agents consistently bypass stated constraints to pursue alternative goals [5]. Prior work [59] also shows that in long-horizon tasks, agent state can drift and remain misaligned even when individual reasoning steps appear locally coherent, and that increasing context capacity alone does not prevent this drift.

Metric misalignment: The reviewer module of the deep research agent checks whether the paper is internally consistent (e.g., methods match results) but does not verify that the reported metrics test the stated objective. This requires a *provenance chain* that links the objective (reduce compute) to the chosen metrics (accuracy only) and to the acceptance decision [48, 62]. Without this linkage, debate-style review cannot surface the error if both agents share the same metric mismatch [24]. The root cause is that the system has no way to remember and enforce the compute constraint throughout the workflow [11, 54]. Worryingly, the authors in [50] find that more capable agents are more prone to misalignment, and that persona characteristics can strongly and unpredictably shape this tendency. Work on scientific integrity and transparency motivates architectures that make these links traceable and accountable [13, 37].

Novelty verification failures: The AI Scientist reports micro-batching for stochastic gradient descent algorithm as a novel contribution [32]. This technique is standard in model training, but